

Trabalho de Redes - 2026.1

Cliente - P2P

Introdução

Cliente de chat P2P implementado em Go para a disciplina de Redes de Computadores. Cada instância (peer) registra-se em um servidor Rendezvous, descobre periodicamente outros peers do mesmo namespace e mantém conexões TCP persistentes com eles para troca de mensagens em tempo real (mensagens diretas, broadcast, keep-alive com medição de RTT e encerramento controlado).

Arquitetura do Sistema:

Visão Geral por Pacote

A seguir, descreve-se a estrutura de pacotes do sistema, detalhando a responsabilidade de cada componente:

- **protocolo**: Define a estrutura de todas as mensagens de comunicação (Rendezvous e P2P) e implementa a função `GerarIDMensagem()`, responsável pela geração de identificadores únicos (UUID v4) para cada mensagem (`msg_id`).
- **configuracao**: Responsável pela gestão de parâmetros do sistema. Inclui a definição da struct `Configuracao`, a função `CarregarConfiguracao` (que processa o arquivo `config.json` e aplica valores padrão) e o método `MeuID()` para identificação local.
- **registro**: Implementa o sistema de logging com suporte a diferentes níveis de severidade (`Depurar`, `Informar`, `Alertar`, `Erro`) e a função `DefinirNivel` para ajuste dinâmico durante a execução (runtime).
- **peer**: Gerencia a comunicação direta entre nós. Composto por `ConexaoPeer` (gerencia a conexão TCP, canais de comunicação, RTT e ACKs pendentes), `GerenciadorConexoes` (mapeamento de conexões ativas) e `TabelaPeers` (catálogo de peers conhecidos obtidos via Rendezvous).
- **rendezvous**: Implementa o `ClienteRendezvous` com operações de `Registrar`, `Descobrir` e `Desregistrar`. Cada operação estabelece uma conexão TCP dedicada. Inclui `IniciarRenovacaoPeriodica` para a manutenção automática do registro (TTL).
- **roteador**: Atua como o núcleo de roteamento de dados. Contém o `Roteador` com os métodos `Enviar` (para protocolos com confirmação via ACK), `Publicar` (PUB), `TratarMensagemRecebida` e `TratarPublicacaoRecebida`.
- **rede**: Camada de transporte e conectividade. Inclui o `Servidor` (gestão de conexões entrantes e handshake), o `Conector` (conexões de saída com lógica de backoff),

rotinas internas como `iniciarLeitor` (loop de processamento de entrada), `iniciarManutencaoConexao` (monitoramento PING/PONG) e `EnviarTchau` para encerramento elegante das sessões (BYE/BYE_OK).

- **descoberta**: Orquestra a atualização da rede através da função `IniciarLoopDescoberta`, que executa o método `Descobrir` a cada 60 segundos, mantendo a `TabelaPeers` atualizada.
- **cli**: Fornece a interface interativa de linha de comando para interação com o usuário.
- **main**: Responsável pelo ponto de entrada da aplicação, orquestração da injeção de dependências e inicialização dos processos do sistema.

Protocolos:

O protocolo Rendezvous é responsável pelo registro dos peers e pela descoberta de outros participantes da rede.

REGISTER:

Quando o cliente é iniciado, ele envia uma mensagem REGISTER ao servidor Rendezvous para informar sua identificação, o namespace ao qual pertence e a porta em que ficará aguardando conexões de outros peers. Caso o registro seja realizado com sucesso, o servidor responde confirmando a operação.

Exemplo de troca de mensagens:

Cliente > Servidor `{"type":"REGISTER","namespace":"CIC","name":"alice","port":4000,"ttl":3600}`

Servidor > Cliente `{"status":"OK","ttl":3600,"ip":"200.1.2.3","port":4000}`

DISCOVER:

Após o registro, o cliente realiza periodicamente uma operação DISCOVER, além de executá-la sempre que o usuário utiliza o comando `/peers`. Essa operação permite obter a lista de peers disponíveis no mesmo namespace.

Exemplo:

Cliente > Servidor `{"type":"DISCOVER","namespace":"CIC"}`

Servidor > Cliente:

`{"status":"OK","peers":[{"ip":"200.1.2.4","port":4001,"name":"bob","namespace":"CIC","expires_in":3412}]}`

Se nenhum namespace for informado, o servidor retorna os peers de todos os namespaces.

UNREGISTER:

Ao encerrar o programa, o cliente envia uma mensagem UNREGISTER para remover seu registro do servidor Rendezvous.

Exemplo:

Cliente > Servidor `{"type":"UNREGISTER","namespace":"CIC","name":"alice","port":4000}`

Servidor > Cliente `{"status":"OK"}`

Além disso, para evitar que o registro expire durante a execução do programa, o cliente envia automaticamente um novo REGISTER em intervalos regulares, renovando seu tempo de vida no servidor.

Protocolo P2P

Depois que os peers são descobertos, toda a comunicação passa a ocorrer diretamente entre eles por meio de conexões TCP persistentes.

HELLO / HELLO_OK:

Assim que uma conexão é estabelecida, ocorre um handshake para que os dois peers se identifiquem e confirmem que estão utilizando a mesma versão do protocolo.

Peer A > Peer B:

`{"type":"HELLO","peer_id":"alice@CIC","version":"1.0","features":["ack","metrics"],"ttl":1}`

Peer B > Peer A:

`{"type":"HELLO_OK","peer_id":"bob@CIC","version":"1.0","features":["ack","metrics"],"ttl":1}`

PING / PONG:

Durante toda a conexão, os peers trocam mensagens PING e PONG para verificar se a conexão continua ativa e medir o tempo de resposta (RTT).

Peer A > Peer B `{"type":"PING","msg_id":"a1b2-...", "timestamp":"2026-06-13T10:00:00Z","ttl":1}`

Peer B > Peer A

`{"type":"PONG","msg_id":"a1b2-...", "timestamp":"2026-06-13T10:00:00Z","ttl":1}`

Caso o cliente deixe de receber a resposta dentro do tempo esperado, a conexão é considerada perdida e é encerrada.

SEND / ACK:

Para enviar uma mensagem diretamente a outro peer, é utilizado o protocolo SEND. Após receber a mensagem, o destinatário responde com um ACK, confirmando que ela foi entregue.

Peer A > Peer B

```
{"type":"SEND","msg_id":"e5f6-...","src":"alice@CIC","dst":"bob@CIC","payload":"oi!","require_ack":true,"ttl":1}
```

Peer B > Peer A

```
{"type":"ACK","msg_id":"e5f6-...","timestamp":"2026-06-13T10:00:01Z","ttl":1}
```

Se essa confirmação não chegar dentro do tempo limite, o cliente registra a falha no log.

PUB:

O protocolo PUB é utilizado para enviar uma mesma mensagem para vários peers ao mesmo tempo, podendo ser destinada a todos os participantes da rede ou apenas aos que pertencem a um determinado namespace.

Exemplo:

```
{"type":"PUB","msg_id":"g7h8-...","src":"alice@CIC","dst":"#CIC","payload":"reunião às 14h","require_ack":false,"ttl":1}
```

ou

```
{"type":"PUB","msg_id":"i9j0-...","src":"alice@CIC","dst":"*","payload":"aviso global","require_ack":false,"ttl":1}
```

Nesse caso, não é enviada nenhuma confirmação de recebimento.

BYE / BYE_OK:

Quando um peer encerra sua participação na rede, ele envia uma mensagem BYE para os peers conectados. O destinatário responde com BYE_OK, indicando que o encerramento da conexão foi realizado corretamente.

Peer A > Peer B

```
{"type":"BYE","msg_id":"k1l2-...","src":"alice@CIC","dst":"bob@CIC","reason":"encerrando sessão","ttl":1}
```

Peer B > Peer A

```
{"type":"BYE_OK","msg_id":"k1l2-...","src":"bob@CIC","dst":"alice@CIC","ttl":1}
```

Instruções para a execução do programa

Requisitos mínimos

- **Sistema operacional:** Linux 64-bit (testado em Ubuntu/Debian)
- **Go 1.21 ou superior**
- **Conectividade:**
 - Acesso à internet/rede para alcançar o servidor Rendezvous.
 - A porta TCP local definida em `porta` (config.json) precisa estar livre e, se houver firewall/NAT entre os peers, liberada/redirecionada para que outros peers consigam abrir conexões de entrada
- **Dependências externas:** [nenhuma](#). O projeto usa exclusivamente a biblioteca padrão do Go.

Instalando o Go

Baixar o binário do Go

```
wget https://go.dev/dl/go1.21.0.linux-amd64.tar.gz
```

Extrair o conteúdo para o diretório /usr/local

```
sudo tar -C /usr/local -xzf go1.21.0.linux-amd64.tar.gz
```

Adicionar o Go ao PATH do sistema

```
echo 'export PATH=$PATH:/usr/local/go/bin' >> ~/.bashrc
```

Aplicar as alterações no shell atual

```
source ~/.bashrc
```

Verifique a instalação:

```
go version
```

```
# go version go1.21.x linux/amd64
```

Como executar

Para executar o sistema, certifique-se de que o arquivo de configuração esteja disponível.

Comandos de Inicialização

A execução padrão utiliza o arquivo `config.json` localizado no mesmo diretório do binário:

```
# Bash
./cliente-p2p
```

Para utilizar um arquivo de configuração específico (caminho ou nome personalizado), utilize a flag `-config`:

```
# Bash
./cliente-p2p -config config-bob.json
```

Encerramento do Sistema

O encerramento do processo pode ser realizado através do comando `/quit` ou enviando os sinais de interrupção `Ctrl+C` (SIGINT/SIGTERM).

1. Envia BYE para cada peer com conexão ativa e aguarda BYE_OK (até 2s por peer);
2. Envia UNREGISTER ao servidor Rendezvous, removendo o registro deste peer;
3. Encerra o processo.

Conclusão:

Tendo em vista os testes realizados e o que foi exigido nas especificações do trabalho, é possível afirmar que os objetivos foram cumpridos e o projeto atende às expectativas propostas. Foi desenvolvida uma aplicação P2P funcional, capaz de registrar peers em um servidor Rendezvous, descobrir outros participantes da rede e estabelecer conexões TCP persistentes para troca de mensagens. Além de atender aos requisitos obrigatórios, a construção do projeto contribuiu para o entendimento de forma prática de conceitos importantes de redes.

